

CircleGuard

Proyecto Final - Ingeniería de Software V

Arquitectura de microservicios con DevOps, seguridad y observabilidad

Autores

Santiago Morales

David A. Troya

Repositorio

<https://github.com/DavidIsGod/Proyecto-Final-Ingesoft-V.git>

Fecha

Junio 2026

Índice

1. Resumen del sistema
2. Arquitectura y microservicios
3. Infraestructura local con Docker Compose
4. CI/CD con GitHub Actions
5. Pruebas automatizadas
6. Kubernetes
7. Terraform
8. Observabilidad y monitoreo
9. Seguridad
10. Gestión de cambios y releases
11. Evidencias de ejecución
12. Problemas encontrados y correcciones
13. Conclusiones

1. Resumen del sistema

CircleGuard es un sistema de gestión de salud universitaria basado en microservicios. El proyecto permite representar flujos relacionados con autenticación, identidad anónima, encuestas de salud, notificaciones, validación de acceso, archivos, promoción de estados y consulta de información operativa.

La entrega se trabajó sobre un monorepo con servicios backend en Spring Boot, una aplicación móvil en Expo/React Native, infraestructura local con Docker Compose, manifiestos de Kubernetes, módulos de Terraform, workflows de GitHub Actions y documentación técnica en la carpeta docs/.

El objetivo principal fue dejar una base ejecutable y sustentable que cubra prácticas de DevOps, seguridad, observabilidad, pruebas automatizadas e infraestructura como código.

2. Arquitectura y microservicios

La arquitectura separa responsabilidades por servicio. Cada microservicio se puede probar y desplegar de forma independiente, apoyándose en componentes de infraestructura como PostgreSQL, Neo4j, Kafka, Zookeeper, Redis y OpenLDAP.

Servicio	Puerto	Responsabilidad principal
circleguard-auth-service	8180	Autenticación, LDAP y JWT
circleguard-notification-service	8082	Notificaciones y eventos
circleguard-identity-service	8083	Identidad anónima y vault
circleguard-dashboard-service	8084	Consulta de métricas y analytics
circleguard-file-service	8085	Gestión de archivos
circleguard-form-service	8086	Encuestas y formularios de salud
circleguard-gateway-service	8087	Validación de acceso y QR
circleguard-promotion-service	8088	Estados de salud y lógica con grafos

El gateway y los manifiestos de Kubernetes permiten enrutar tráfico hacia los servicios. La comunicación se apoya en REST y eventos Kafka cuando aplica.

3. Infraestructura local con Docker Compose

Para el ambiente local se usa docker-compose.dev.yml. Este archivo levanta los componentes de soporte necesarios para ejecutar los servicios backend.

```
docker compose -f docker-compose.dev.yml up -d
docker compose -f docker-compose.dev.yml ps
```

En la validación realizada se levantaron 6 contenedores base: PostgreSQL, Neo4j, Kafka, Zookeeper, Redis y OpenLDAP.

```

morales@MacBook-Air-de-Santiago-2 Proyecto-Final-Ingesoft-V % docker compose -f docker-compose.dev.yml up -d
docker compose -f docker-compose.dev.yml ps
WARN[0000] Found orphan containers ([proyecto-final-ingesoft-v-fluent-bit-1 proyecto-final-ingesoft-v-kibana-1 proyect
o-final-ingesoft-v-jaeger-1 proyecto-final-ingesoft-v-prometheus-1 proyecto-final-ingesoft-v-grafana-1 proyecto-final-
ingesoft-v-elasticsearch-1]) for this project. If you removed or renamed this service in your compose file, you can ru
n this command with the --remove-orphans flag to clean it up.
[+] up 6/6
✓ Container circleguard-neo4j Started 0.2s
✓ Container circleguard-postgres Started 0.2s
✓ Container circleguard-zookeeper Started 0.2s
✓ Container circleguard-redis Started 0.1s
✓ Container circleguard-ldap Started 0.2s
✓ Container circleguard-kafka Started 0.2s
NAME IMAGE COMMAND SERVICE CREATED STATUS
TUS PORTS
circleguard-kafka confluentinc/cp-kafka:7.6.0 "/etc/confluent/dock... kafka About an hour ago Up
Less than a second 0.0.0.0:9092->9092/tcp, [::]:9092->9092/tcp
circleguard-ldap osixia/openldap:1.5.0 "/container/tool/run" openldap About an hour ago Up
Less than a second 0.0.0.0:389->389/tcp, [::]:389->389/tcp, 0.0.0.0:636->636/tcp, [::]:636->636/tcp
circleguard-neo4j neo4j:5.26 "tiny -g -- /startup..." neo4j About an hour ago Up
Less than a second 0.0.0.0:7474->7474/tcp, [::]:7474->7474/tcp, 0.0.0.0:7687->7687/tcp, [::]:7687->7687/tcp
circleguard-postgres postgres:16 "docker-entrypoint.s..." postgres About an hour ago Up
Less than a second 0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
circleguard-redis redis:7.2 "docker-entrypoint.s..." redis About an hour ago Up
Less than a second 0.0.0.0:6379->6379/tcp, [::]:6379->6379/tcp
circleguard-zookeeper confluentinc/cp-zookeeper:7.6.0 "/etc/confluent/dock... zookeeper About an hour ago Up
Less than a second 2181/tcp, 2888/tcp, 3888/tcp

```

Figura 1. Infraestructura local levantada con Docker Compose.

4. CI/CD con GitHub Actions

El proyecto usa GitHub Actions para automatizar tareas de integración continua. Los workflows están en .github/workflows/ y cubren validación backend, mobile, Terraform, escaneo de seguridad con Trivy, releases y despliegues dry-run para Kubernetes.

Durante la validación se corrigieron errores de workflows, incluyendo fallos en la versión de Trivy, dry-run de Kubernetes sin clúster real, pruebas mobile y diferencias entre ejecución local y CI.

```

MacBook-Air-de-Santiago-2:Proyecto-Final-Ingesoft-V morales$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
MacBook-Air-de-Santiago-2:Proyecto-Final-Ingesoft-V morales$ git log --oneline --decorate -5
f5812bf (HEAD -> main, origin/main, origin/HEAD) corregir fallos de github actions
1d6cbec corregir health check de notification y actualizar evidencia
93ac2a0 validar entrega final con evidencia y documentación
6652c2c implementar base devops del proyecto final
52e4755 Merge branch 'referencia/master' into main

```

Figura 2. Estado del repositorio y últimos commits relacionados con la entrega.

El repositorio quedó limpio y sincronizado con origin/main después de los ajustes.

5. Pruebas automatizadas

5.1 Pruebas backend y cobertura

Las pruebas backend se ejecutan con Gradle. El comando también genera reportes de cobertura con JaCoCo.

```
./gradlew test jacocoTestReport
```

La ejecución local terminó en BUILD SUCCESSFUL. Esto confirma que las pruebas disponibles pasan en el entorno local usado para la entrega.

```

morales@MacBook-Air-de-Santiago-2 Proyecto-Final-Ingesoft-V % ./gradlew test jacocoTestReport
openjdk version "17.0.19" 2026-04-21
OpenJDK Runtime Environment Homebrew (build 17.0.19+0)
OpenJDK 64-Bit Server VM Homebrew (build 17.0.19+0, mixed mode, sharing)
Starting a Gradle Daemon, 1 busy and 2 incompatible Daemons could not be reused, use --status for details

[Incubating] Problems report is available at: file:///Users/morales/Desktop/Proyecto-Final-Ingesoft-V/build/reports/problems/problems-report.html

Deprecated Gradle features were used in this build, making it incompatible with Gradle 9.0.

You can use '--warning-mode all' to show the individual deprecation warnings and determine if they come from your own scripts or plugins.

For more on this, please refer to https://docs.gradle.org/8.14/userguide/command_line_interface.html#sec:command_line_warnings in the Gradle documentation.

BUILD SUCCESSFUL in 11s
55 actionable tasks: 9 executed, 46 up-to-date

```

Figura 3. Ejecución local de pruebas backend con BUILD SUCCESSFUL.

```

morales@MacBook-Air-de-Santiago-2 Proyecto-Final-Ingesoft-V % ./gradlew test jacocoTestReport
openjdk version "17.0.19" 2026-04-21
OpenJDK Runtime Environment Homebrew (build 17.0.19+0)
OpenJDK 64-Bit Server VM Homebrew (build 17.0.19+0, mixed mode, sharing)

[Incubating] Problems report is available at: file:///Users/morales/Desktop/Proyecto-Final-Ingesoft-V/build/reports/problems/problems-report.html

Deprecated Gradle features were used in this build, making it incompatible with Gradle 9.0.

You can use '--warning-mode all' to show the individual deprecation warnings and determine if they come from your own scripts or plugins.

For more on this, please refer to https://docs.gradle.org/8.14/userguide/command_line_interface.html#sec:command_line_warnings in the Gradle documentation.

BUILD SUCCESSFUL in 2s
55 actionable tasks: 9 executed, 46 up-to-date

```

Figura 4. Segunda evidencia de ejecución de pruebas y generación de cobertura.

5.2 Health checks

Los servicios exponen endpoints de Actuator para health, liveness y readiness. Se validaron los puertos principales del sistema con curl.

```

curl http://localhost:8180/actuator/health
curl http://localhost:8082/actuator/health
curl http://localhost:8083/actuator/health
curl http://localhost:8084/actuator/health
curl http://localhost:8085/actuator/health
curl http://localhost:8086/actuator/health
curl http://localhost:8087/actuator/health
curl http://localhost:8088/actuator/health

```

La respuesta de los servicios fue UP, incluyendo los grupos liveness y readiness.

```

morales@MacBook-Air-de-Santiago-2 Proyecto-Final-Ingesoft-V % curl http://localhost:8180/actuator/health
curl http://localhost:8082/actuator/health
curl http://localhost:8083/actuator/health
curl http://localhost:8084/actuator/health
curl http://localhost:8085/actuator/health
curl http://localhost:8086/actuator/health
curl http://localhost:8087/actuator/health
curl http://localhost:8088/actuator/health
{"status":"UP","groups":{"liveness","readiness"}}{"status":"UP","groups":{"liveness","readiness"}}{"status":"UP","groups":{"liveness","readiness"}}{"status":"UP","groups":{"liveness","readiness"}}{"status":"UP","groups":{"liveness","readiness"}}{"status":"UP","groups":{"liveness","readiness"}}{"status":"UP","groups":{"liveness","readiness"}}{"status":"UP","groups":{"liveness","readiness"}}{"status":"UP","groups":{"liveness","readiness"}}

```

Figura 5. Validación de endpoints /actuator/health en los servicios.

5.3 Pruebas E2E, rendimiento y seguridad

El repositorio incluye pruebas complementarias en la carpeta tests/. Se dejaron configurados flujos smoke E2E, pruebas de rendimiento con Locust y baseline de seguridad con OWASP ZAP.

```
./tests/e2e/smoke.sh
locust -f tests/locust/locustfile.py --host http://localhost:8180
./tests/security/zap-baseline.sh
```

Estas pruebas sirven como soporte para validar el sistema más allá de las pruebas unitarias. En el informe se documenta su configuración y forma de ejecución.

6. Kubernetes

La carpeta k8s/ contiene la configuración para desplegar los servicios y componentes necesarios en Kubernetes. Incluye namespace, deployments, services, ConfigMaps, secrets de ejemplo, ingress, RBAC y probes de liveness/readiness.

```
kubectl apply -k k8s/ --dry-run=client --validate=false -o yaml
```

El despliegue real requiere un clúster configurado. Para la entrega se validó el dry-run para comprobar que los manifiestos se pueden procesar sin depender de un clúster local activo.

7. Terraform

La infraestructura como código se organizó en terraform/. El proyecto maneja módulos y ambientes separados para dev, stage y prod.

Elemento	Descripción
Módulos	network, database y kubernetes
Ambientes	dev, stage y prod
Estado remoto	backend S3 de ejemplo documentado
Validación	terraform init -backend=false y terraform validate

En GitHub Actions se evidenció que el job de Terraform pasó correctamente.

8. Observabilidad y monitoreo

El stack de observabilidad se levanta con docker-compose.observability.yml. Incluye Prometheus, Grafana, Jaeger, Elasticsearch, Kibana y Fluent Bit.

```
docker compose -f docker-compose.observability.yml up -d
docker compose -f docker-compose.observability.yml ps
```

La validación local mostró los contenedores del stack en estado Up.

```
morales@MacBook-Air-de-Santiago-2 Proyecto-Final-Ingesoft-V % docker compose -f docker-compose.observability.yml up -d
docker compose -f docker-compose.observability.yml ps
WARN[0000] Found orphan containers ([circleguard-kafka circleguard-postgres circleguard-redis circleguard-zookeeper ci
rcleguard-neo4j circleguard-ldap]) for this project. If you removed or renamed this service in your compose file, you
can run this command with the --remove-orphans flag to clean it up.
[+] up 6/6
✓ Container projecto-final-ingesoft-v-grafana-1 Started 0.2s
✓ Container projecto-final-ingesoft-v-prometheus-1 Started 0.1s
✓ Container projecto-final-ingesoft-v-jaeger-1 Started 0.1s
✓ Container projecto-final-ingesoft-v-elasticsearch-1 Started 0.2s
✓ Container projecto-final-ingesoft-v-fluent-bit-1 Started 0.2s
✓ Container projecto-final-ingesoft-v-kibana-1 Started 0.2s
NAME SERVICE CREATED STATUS PORTS COMMAND
circleguard-ldap n" openldap About an hour ago Up 52 seconds osixia/openldap:1.5.0 "/container/tool/ru
->636/tcp, [::]:636->636/tcp
circleguard-neo4j p..." neo4j About an hour ago Up 52 seconds neo4j:5.26 "tini -g -- /startu
:7687->7687/tcp, [::]:7687->7687/tcp
circleguard-postgres s..." postgres About an hour ago Up 52 seconds postgres:16 "docker-entrypoint.
circleguard-redis s..." redis About an hour ago Up 52 seconds redis:7.2 "docker-entrypoint.
circleguard-zookeeper k..." zookeeper About an hour ago Up 52 seconds confluentinc/cp-zookeeper:7.6.0 "/etc/confluent/doc
projecto-final-ingesoft-v-elasticsearch-1 l..." elasticsearch About an hour ago Up Less than a second docker.elastic.co/elasticsearch/elasticsearch:8.12.0 "/bin/tini -- /usr/
projecto-final-ingesoft-v-fluent-bit-1 u..." fluent-bit About an hour ago Up Less than a second fluent/fluent-bit:2.2 0.0.0.0:9200->9200/tcp, [::]:9200->9200/tcp "/fluent-bit/bin/fl
projecto-final-ingesoft-v-grafana-1 grafana About an hour ago Up Less than a second grafana/grafana:10.4.0 2020/tcp "/run.sh"
projecto-final-ingesoft-v-jaeger-1 -..." jaeger About an hour ago Up Less than a second jaegertracing/all-in-one:1.56 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp "/go/bin/all-in-one
:16686->16686/tcp, [::]:16686->16686/tcp
projecto-final-ingesoft-v-kibana-1 kibana About an hour ago Up Less than a second docker.elastic.co/kibana/kibana:8.12.0 "/bin/tini -- /usr/
projecto-final-ingesoft-v-prometheus-1 c..." prometheus About an hour ago Up Less than a second prom/prometheus:v2.51.0 0.0.0.0:5601->5601/tcp, [::]:5601->5601/tcp "/bin/prometheus --
0.0.0.0:9090->9090/tcp, [::]:9090->9090/tcp
```

Figura 6. Stack de observabilidad levantado con Docker Compose.

Prometheus quedó disponible en el puerto 9090 para consultar métricas.

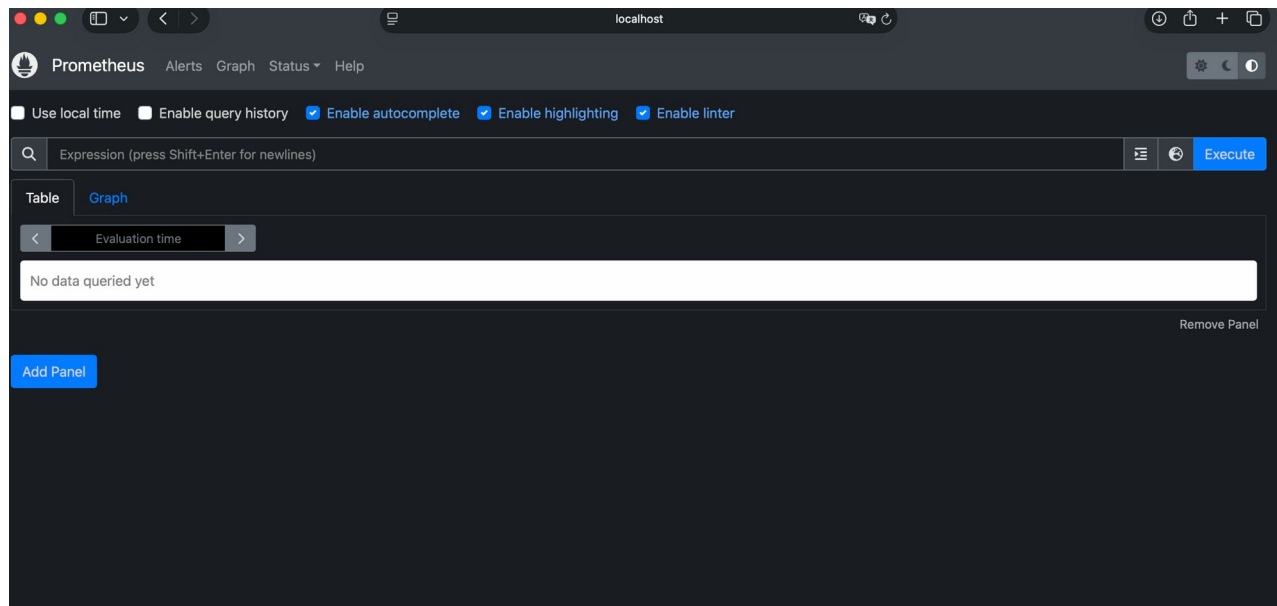


Figura 7. Prometheus disponible en localhost:9090.

Grafana quedó disponible en el puerto 3000 para visualizar dashboards.

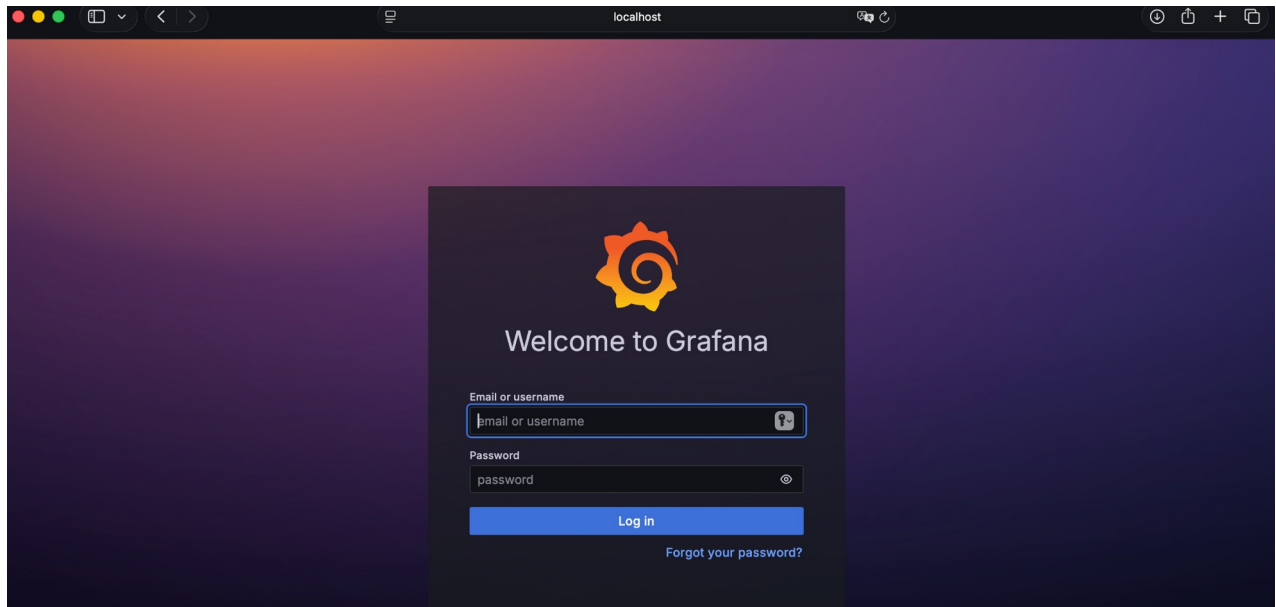


Figura 8. Pantalla de acceso a Grafana.

Jaeger quedó disponible en el puerto 16686 para trazas distribuidas. La instrumentación completa de OpenTelemetry queda como trabajo posterior, pero el componente está desplegado.

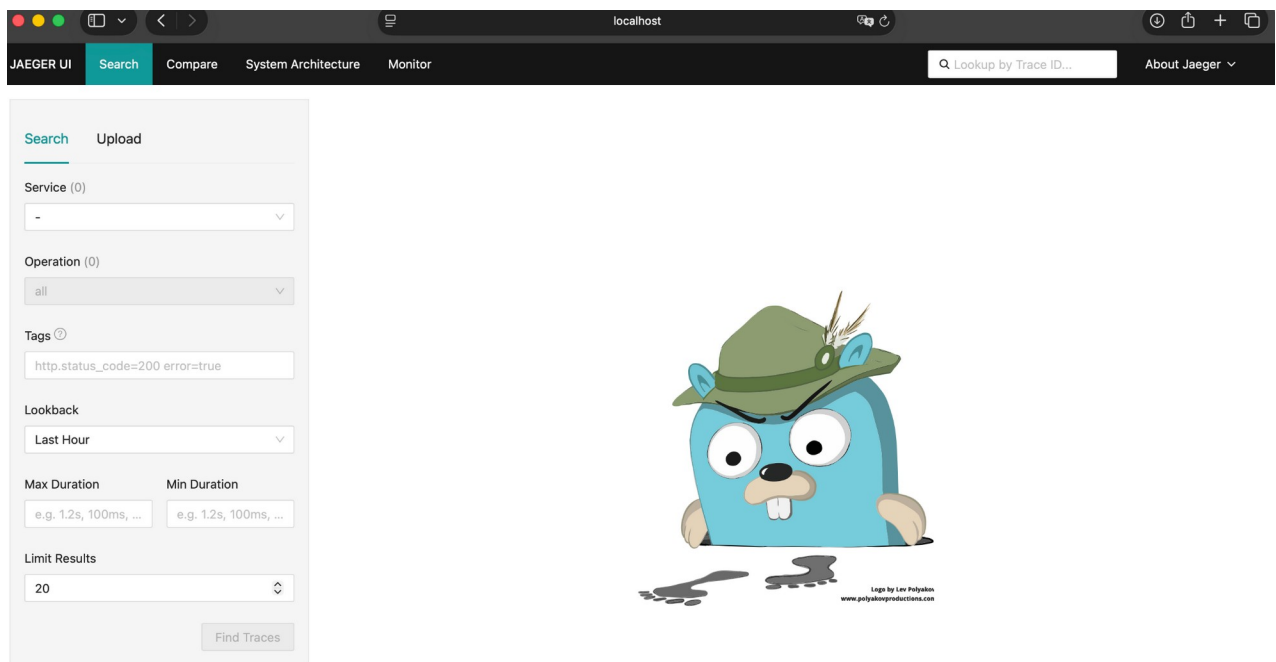


Figura 9. Jaeger UI disponible en localhost:16686.

Kibana quedó disponible en el puerto 5601 como interfaz para explorar logs en Elasticsearch.

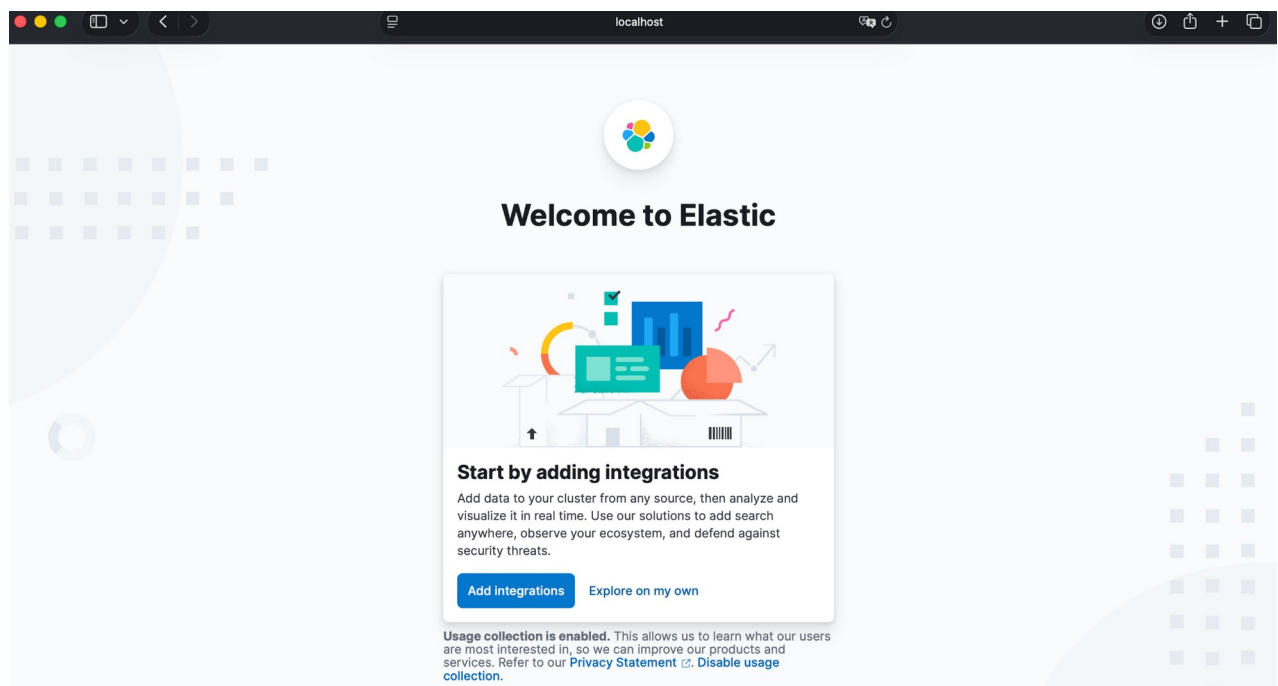


Figura 10. Kibana disponible en localhost:5601.

9. Seguridad

La seguridad del proyecto se trabaja desde varias capas: autenticación con JWT, configuración de secretos de ejemplo, RBAC en Kubernetes, TLS de ejemplo en ingress y escaneo de imágenes con Trivy en GitHub Actions.

Aspecto	Implementación
Autenticación	JWT y LDAP en ambiente de desarrollo
Secretos	Archivos de ejemplo, sin credenciales reales
Kubernetes	ServiceAccount, Role y RoleBinding
TLS	Configuración de ejemplo para ingress
Escaneo	Trivy en workflow de seguridad

También se dejó un baseline de OWASP ZAP para revisar endpoints HTTP durante pruebas de seguridad.

10. Gestión de cambios y releases

El proyecto incluye documentación de change management, CHANGELOG y RELEASE_NOTES. Además, los workflows permiten generar releases y notas a partir de cambios versionados.

Los últimos commits reflejan correcciones de health checks, evidencias, documentación y GitHub Actions. Esto deja un historial claro para sustentar qué se fue ajustando durante la entrega.

11. Evidencias de ejecución

Las principales evidencias obtenidas durante la validación fueron:

Validación	Resultado observado
git status	Repositorio limpio y actualizado con origin/main
docker compose dev	6/6 contenedores de infraestructura en estado Up
./gradlew test jacocoTestReport	BUILD SUCCESSFUL en local
docker compose observability	Prometheus, Grafana, Jaeger, Elasticsearch, Kibana y Fluent Bit en estado Up
Health endpoints	Servicios respondiendo status UP

12. Problemas encontrados y correcciones

Durante la preparación de la entrega se encontraron algunos problemas puntuales. El más relevante fue notification-service: el servicio sí levantaba, pero el health general aparecía DOWN porque Spring intentaba validar correo contra localhost:25 y no había servidor SMTP local.

La solución fue desactivar el indicador de salud de mail para este entorno en la configuración de Actuator:

```
management:
  health:
    mail:
      enabled: false
```

También se corrigieron workflows de GitHub Actions para que no dependieran de secretos reales ni de un clúster Kubernetes activo cuando solo se estaba haciendo validación dry-run.

13. Conclusiones

El proyecto quedó organizado como una entrega completa de microservicios con soporte de DevOps. Se validó la infraestructura local, las pruebas backend, los endpoints de salud, la observabilidad y la estructura de Kubernetes y Terraform.

La automatización con GitHub Actions permite revisar pruebas, seguridad, Terraform y despliegues de forma controlada. Para producción real todavía haría falta configurar un clúster Kubernetes, secretos reales, dominio, certificados y variables de GitHub Environments.

La arquitectura facilita explicar el sistema por partes: servicios, infraestructura, pruebas, seguridad y monitoreo. Esto ayuda a sustentar el proyecto sin depender solo del código.